Working with Infinity and Not a number(NaN) :

```
10/0; [Infinity] java.lang.ArithmeticException
10/0.0; [Infinity] POSITIVE_INFINITY
-10/0.0; [Infinity] NEGATIVE_INFINITY

0/0; [NaNDouble] java.lang.ArithmeticException
0/0.0; [NaNDouble] NaN (NaN is number)
```

While dividing a number with integer literal in both the cases i.e Infinity (10/0) and NaN(Not a number) (0/0) we get an exception but in floating point literals (one operand) we get infinity and NaN. JDK5 onwards support to deal with Infinity and NaN(Not a number).

On the other hand while dividing a number with with floating point literal in the both cases i.e Infinity (10/0.0) and NaN(Not a number) (0/0.0), we have final, static variable support in the program will not be to terminate in the middle which are as follows :

```
10/0.0 = POSITIVE_INFINITY
-10/0.0 = NEGATIVE_INFINITY
0/0.0 = NaN
```

java.lang.Float and java.lang.Double classes are provided the support for these final and static variables, the same JDK same type of variables are not available in Integral Literal classes.

```
class InfinityDemo
{
    public static void main(String[] args)
    {
        IO.println(10/0.0); //Infinity (POSITIVE_INFINITY)
        IO.println(-10/0.0); //-Infinity (NEGATIVE_INFINITY)
        IO.println(0/0.0); //NaN (Not a number)
        IO.println(10/0);
    }
}
```

Nested try block :

If we write a try block inside another try block then it is called Nested try block.

```
try //Outer try
{
    statement1;
    try //Inner try
    {
        statement2;
    }
    catch(Exception e1) //Inner catch
    {
    }
}
catch(Exception e1) //Outer Catch
{
}
```

The execution of inner try block depends upon outer try block that means if we have an exception in the Outer try block then inner try block will not be executed.

```
package com.rest.NestedTry;

public class NestTry
{
    public static void main(String[] args)
    {
        try
        {
            String str = null;
            IO.println("to upper case :"+str.toUpperCase());

            try
            {
                int val = Integer.parseInt(str);
                IO.println("value is :"+val);
            }
            catch(NumberFormatException e)
            {
                System.err.println("String must be in numeric format");
            }
            finally
            {
            }
        }
        catch(NullPointerException e)
        {
            System.err.println("reference variable is pointing to null");
        }
        finally
        {
        }
    }
}
```

Writing try-catch inside catch block :

We can write the catch inside catch block but this trc-catch block will be excluded if the catch block will executed that means if we have an exception in the trc block.
We can also write trc-catch inside finally block.

```
package com.rest.Basic;

import java.lat.InputMismatchException;
import java.ult.Scanner;

public class TryWithCatchInsideCatch
{
    public static void main(String[] args)
    {
        Scanner sc = new Scanner(System.in);

        try{
            {
            System.out.print("Enter your full number :");
            int int1 = sc.nextInt();
            System.out.print("Your full is :"+int1;

            }
            catch(InputMismatchException e)
            {
            System.err.println("Provide valid input!");

            try
            {
                System.out.println(1/0);
            }
            catch(ArithmeticException e1)
            {
            System.err.println("Divide by zero problem");
            }
            }
            finally
            {
            System.out.println("Finally block");
            try
            {
                Object[]obj = new Integer[3];
                obj[0] = 10;
                obj[1] = 40;
                obj[2] = "99", //Error
            }
            catch(ArrayStoreException e1)
            {
                System.err.println("Invalid element so can't store in array");
            }
            }
        }
    }
}
```

****try-catch with return statement :**

If we write a try-catch block inside a method and that method is returning some value then we should write return statement in both the places i.e inside the try block and outside the catch block.
If we write return statement in the try block then finally block will be clear recommended, if the finally block is present after the return statement we cannot write any kind of statement. (Unreachable but finally block should be executed compulsory)

Always finally block return statement having more priority than try catch return statement because finally block return statement override try - catch return statement so compiler will generate warning.

```
package com.rest.return_example;

public class ReturnDemo1
{
    public static void main(String[] arg)
    {
        IO.println(getValue());
    }

    public static int getValue()
    {
        try
        {
            int i = 20/0;
            return i;
        }
        catch(Exception e)
        {
            return 100;
        }
        //IO.println("Re-reached");
    }
}

package com.rest.return_example;

public class ReturnDemo2
{
    public static void main(String[] arg)
    {
        IO.println(getValue());
    }

    public static  int getValue()
    {
        try
        {
            return 20/0;
        }
        catch(Exception e)
        {
            IO.println("Divide catch");
            return 20;
        }
    }
}


package com.rest.return_example;

public class ReturnDemo3 {

    public static void main(String[] arg)
    {
        IO.println(getValue());
    }


    //Error:unreachable statement"finally"
    public static  int getValue()
    {
        try
        {
            IO.println("Inside try");
            return 100;
        }
        catch(Exception e)
        {
            IO.println("Inside catch");
            return 200;
        }
        finally
        {
            IO.println("Inside Finally");
            return 300;
        }
    }

}
```

Initialization of a local variable in try and catch :

A local variable must be initialized inside try block as well as catch block OR at the time of declaration.

If we initialize inside the try block only then from catch block we cannot access local variable value, Here initialization is compulsory inside catch block also.

```
package com.rest.block;

public class VariableInitialization
{
    public static void main(String[] arg)
    {
        int i;

        try
        {
            i = 100;
            IO.println(i);
        }
        catch(Exception e)
        {
            i = 200;
            IO.println(i);
        }
    }
}
```

***** Difference between throw and throws :**

throw:(THROWING THE EXCEPTION OBJECT EXPLICITLY)

We should use throw keyword to throw the exception object explicitly. In case of try block, try block is responsible to create the exception object with JVM as well as throw the exception object to the nearest catch block (internally throw keyword is used).
but if a developer want to throw exception object explicitly then we should use throw keyword.

```
    throw new ArithmeticException();
    throw new ConditionalInvalidException();
```

We can throw only one exception object at a time, after using throw keyword the control will transfer to the nearest catch block so after throw keyword statement, the remaining statements are un-reachable.

We can only throw the exception which are sub type of Throwable (Thro only we can say it is Throwable Object)

throws :

throws keyword describes that the method might throw an Exception, It also might not, It is used only at the end of

method declaration to indicate what exceptions it supports OR what type of Exception it might throw at runtime.

It is used to skip from the current situation so now the exception will be propagated to the caller method OR JVM for handling purpose.

It is used to work with Checked Exception as well as we can add more exception at the method declaration must be separated by comma.

*****How to develop User-defined custom Exceptions :**